

Lecture 6 – Logistic Regression

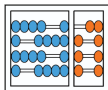
[ISLP 2.2.3, 4.1–4.3]

MC886 – Machine Learning

Marcelo S. Reis ^{1,2}

¹ Departamento de Sistemas de Informação
Instituto de Computação, Universidade Estadual de Campinas

Campinas, March 11, 2026



**Instituto de
Computação**

UNIVERSIDADE ESTADUAL DE CAMPINAS



UNICAMP

Today's summary

The classification setting, Bayes and KNN classifiers

The logistic model

Multinomial (multiclass) logistic regression

Solving the logistic regression cost function

An overview of classification

1. A person arrives at the emergency room with a set of symptoms that could possibly be attributed to one of three medical conditions. Which of the three conditions does the individual have?
2. An online banking service must be able to determine whether or not a transaction being performed on the site is fraudulent, on the basis of the user's IP address, past transaction history, and so forth.
3. On the basis of DNA sequence data for a number of patients with and without a given disease, a biologist would like to figure out which DNA mutations are deleterious (disease-causing) and which are not.

The classification setting

- ❖ We still have a dataset $\{(x_1, y_1), \dots, (x_n, y_n)\}$, where (x_i, y_i) is sampled in a process that makes use of a joint probability distribution $Pr(X, Y)$
- ❖ However, $y_1, \dots, y_n$ are now qualitative
- ❖ How do we measure now the accuracy of our predictions?

Error rate

- ❖ The proportion of mistakes using $\hat{f}$:

$$\frac{1}{n} \sum_{i=1}^n \mathbb{I}(y_i \neq \hat{y}_i), \quad (2.8)$$

where:

- $\hat{y}_i$ is the predicted class for x_i using $\hat{f}$
- $\mathbb{I}(y_i \neq \hat{y}_i) = \begin{cases} 1, & \text{if } y_i \neq \hat{y}_i; \\ 0, & \text{otherwise.} \end{cases}$

Training / test error rates

- ❖ We use Eq. 2.8 to compute the error based on the data used to train the classifier (*train error rate*)
- ❖ The *test error rate* is computed based on observations (x_0, y_0) unseen in training data:

$$\text{Ave}(\mathbb{I}(y_0 \neq \hat{y}_0)), \quad (2.9)$$

where $\hat{y}_0$ is the predicted class for x_0 using $\hat{f}$

- ❖ A good classifier minimizes Eq. 2.8 or 2.9? (or both?)

The Bayes classifier

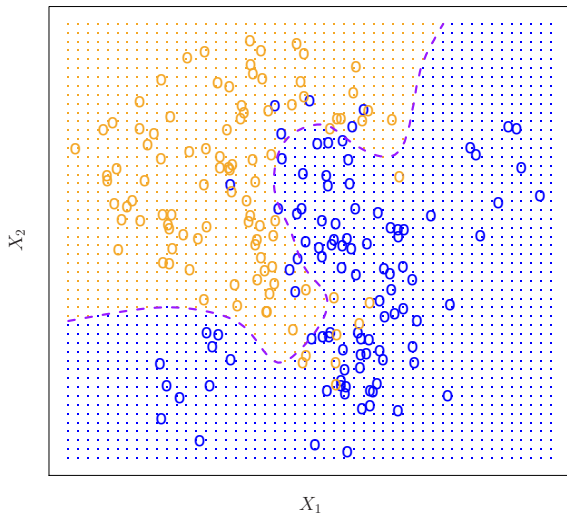
- ❖ To assign each observation to the most likely class
- ❖ That is, we classify a test x_0 vector to the class j such that

$$Pr(Y = j|X = x_0) \quad (2.10)$$

is the largest

- ❖ This classifier is known as the *Bayes classifier*
- ❖ **In the binary case:** $\hat{y}_0 = \begin{cases} 1, & \text{if } Pr(Y = 1|X = x_0) > 0.5, \\ 0, & \text{otherwise.} \end{cases}$

Example: Simulated binary dataset



Purple dashed line is the Bayes decision boundary

Bayes error rate

- ❖ Since the Bayes classifier always choose the class for which Eq. 2.10 is the largest, the overall test error rate (Bayes error rate) is:

$$1 - E \left(\max_j Pr(Y = j|X) \right), \quad (2.11)$$

where the expectation averages the probability of all possible values of X

- ❖ In the example, since there is class overlap in the true population, $\max_j Pr(Y = j|X = x_0) < 1$ for some x_0

The K-Nearest Neighbors (KNN) classifier

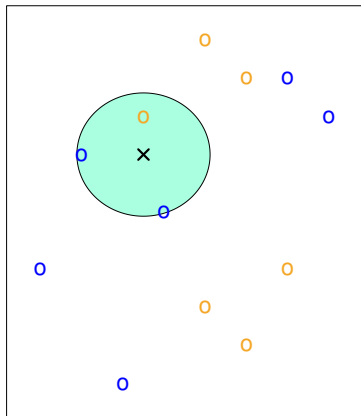
- ❖ However, in practical situations, **we do not have** $Pr(Y|X)$!!!
- ❖ Therefore, for a given point x_0 , we estimate $Pr(Y|X = x_0)$ using K points in the training set that are *nearest* to x_0 :

$$Pr(Y = j|X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} \mathbb{I}(y_i = j), \quad (2.12)$$

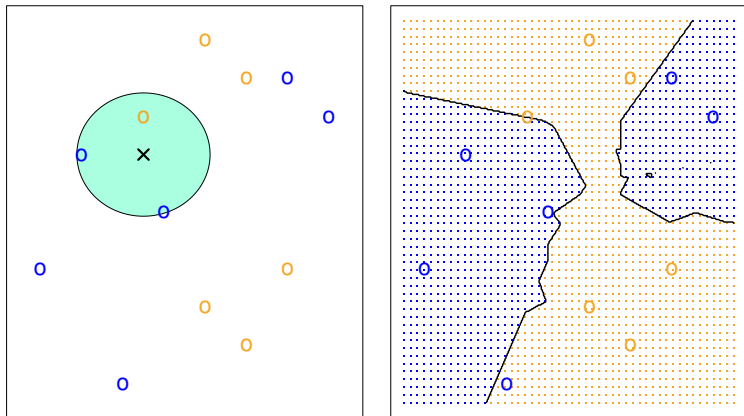
where $\mathcal{N}_0$ are the K nearest points to x_0 in the training data

- ❖ KNN classifies x_0 with the class j with largest probability in Eq. 2.12

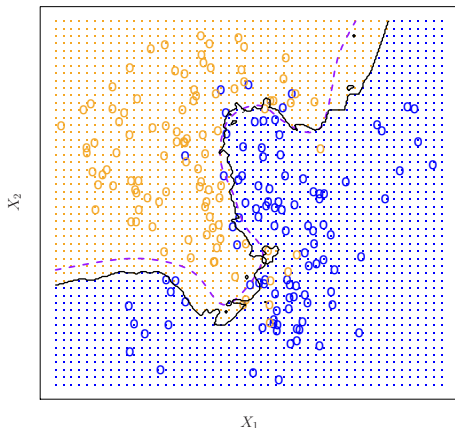
Executing KNN on simulated data ($Pr(X, Y)$ is known),
with $n = 12$ and $K = 3$



Executing KNN on simulated data ($Pr(X, Y)$ is known),
with $n = 12$ and $K = 3$



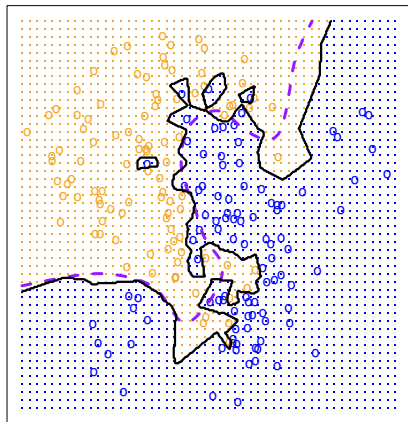
Executing KNN on simulated data ($Pr(X, Y)$ is known),
with larger n and $K = 10$



Purple dashed line is the Bayes decision boundary
Black solid line is the KNN decision boundary

A trade off in KNN

KNN: $K=1$

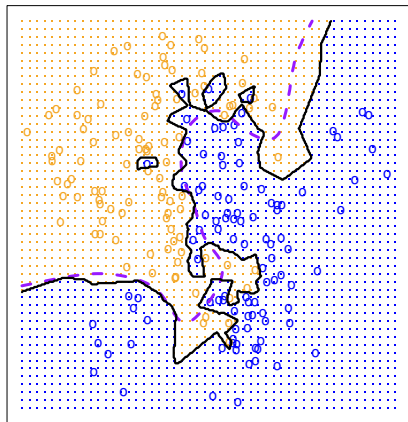


Purple dashed line is the Bayes decision boundary

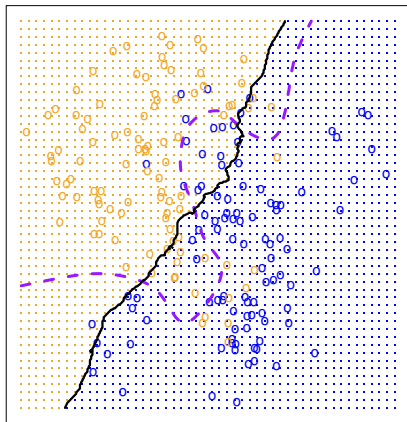
Black solid line is the KNN decision boundary

A trade off in KNN

KNN: $K=1$



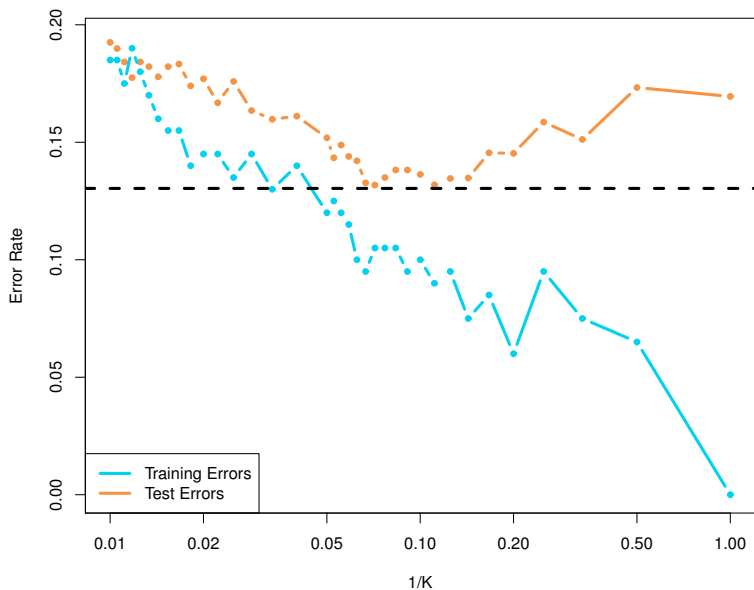
KNN: $K=100$



Purple dashed line is the Bayes decision boundary

Black solid line is the KNN decision boundary

Model complexity and under- and overfitting



And how about linear regression for classification?

- ❖ Consider diagnosing a patient based on her symptoms

$$Y = \begin{cases} 1, & \text{if stroke;} \\ 2, & \text{if drug overdose;} \\ 3, & \text{if seizure} \end{cases}$$

- ❖ This type of coding implies an ordering on the outcomes!
- ❖ We'll deal with this case later

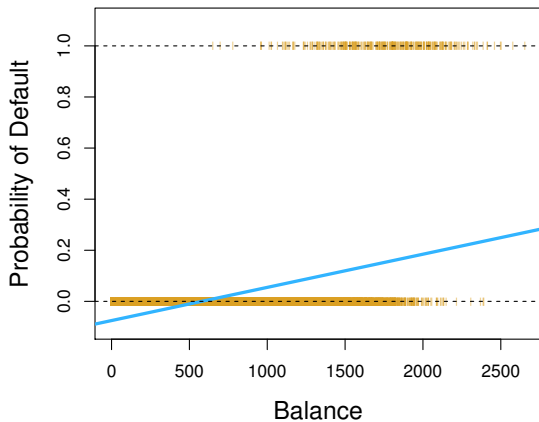
And how about linear regression for classification?

- ❖ Consider now the simpler (binary) case:

$$Y = \begin{cases} 0, & \text{if stroke;} \\ 1, & \text{if drug overdose} \end{cases}$$

- ❖ Here we can use linear regression to obtain an estimate of $Pr(\text{drug overdose}|X)$
- ❖ However, the above estimate might be outside the $[0, 1]$ interval (hard to interpret as probabilities)

Why not linear regression?



Logistic regression

It models the response Y as the *probability* that Y belongs to a particular category. For example:

$$\begin{aligned} Y &= \Pr(\text{default} = \text{Yes} | \text{balance}) \\ &= p(\text{balance}) \end{aligned} \quad (\text{abbreviation})$$

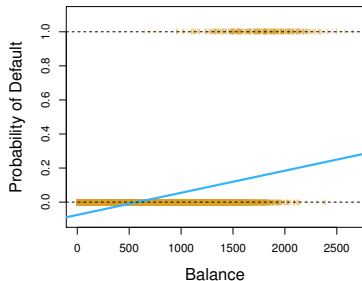
We can then set a threshold to predict $\text{default} = \text{Yes}$:

$$\diamond p(\text{balance}) > 0.5 \iff \text{default} = \text{Yes}$$

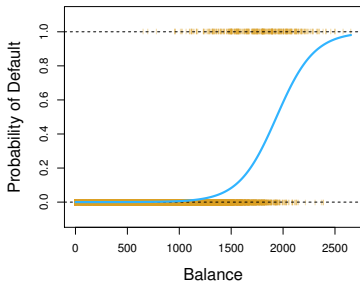
$$\diamond p(\text{balance}) > 0.1 \iff \text{default} = \text{Yes}$$

(more conservative)

Linear regression (left) vs. logistic regression (right)



$$p(X) = \beta_0 + \beta_1 X$$



$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$

The logistic model

To keep the predictions between 0 and 1, we use the *logistic function*:

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}} \quad (4.2)$$

- ❖ How to make the best fit of β to data?
- ❖ **Idea:** the best β would be one that, for each point x_i , it yields a $p(x_i)$ as close as possible to the actual y_i
- ❖ To this end, we'll employ the **maximum likelihood** approach, which will be discussed later in this lecture

Logistic regression on the **balance** variable

	Coefficient value
Intercept (β_0)	-10.6513
balance (β_1)	0.0055

What is the estimated default probability for someone with a balance of \$ 1000?

$$\hat{p}(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}} = \frac{e^{-10.6513 + 0.0055 \times 1000}}{1 + e^{-10.6513 + 0.0055 \times 1000}} = 0.006$$

And for someone with a balance of \$ 5000?

Logistic regression on the **student [Yes]** (0-1) variable

	Coefficient value
Intercept (β_0)	-3.5041
student [Yes] (β_1)	0.4049

$$\hat{\text{Pr}}(\text{default} = \text{Yes} | \text{student} = \text{Yes}) = \frac{e^{-3.5041 + 0.4049 \times 1}}{1 + e^{-3.5041 + 0.4049 \times 1}} = 0.0431$$

$$\hat{\text{Pr}}(\text{default} = \text{Yes} | \text{student} = \text{No}) = \frac{e^{-3.5041 + 0.4049 \times 0}}{1 + e^{-3.5041 + 0.4049 \times 0}} = 0.0292$$

Multiple logistic regression

Similar as in linear regression, we can generalize Eq. 4.2 to have a binary response with multiple variables:

$$p(X) = \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}} \quad (4.7)$$

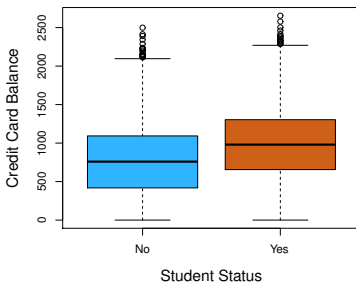
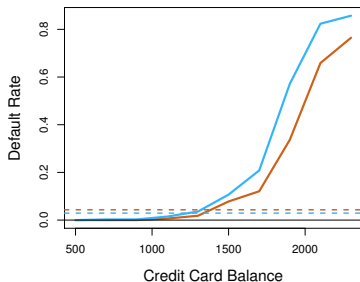
- ❖ As in the simple logistic regression, we estimate $\beta_0, \beta_1, \dots, \beta_p$ using the maximum likelihood approach

Multiple logistic regression on the Default data

	Coefficient value
Intercept (β_0)	-10.8690
balance (β_1)	0.0057
income (β_2)	0.0030
student [Yes] (β_3)	-0.6468

student [Yes] here has a negative coefficient... Why?

Confounding in the Default data



- ❖ Students often have higher balances than non-students (right graph), so the former's marginal default rate (orange dashed line) is higher than the latter's one (blue dashed line)
- ❖ However *for a fixed value* of balance and income, students default less than non-students (orange and blue solid lines)

Multinomial (multiclass) classification

- ❖ For problems with K classes ($K > 2$), we employ the *multinomial logistic regression*, which can be formulated using the **softmax** function:

$$Pr(Y = k|X = x) = \frac{e^{\beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p}}{\sum_{l=1}^K e^{\beta_{l0} + \beta_{l1}x_1 + \dots + \beta_{lp}x_p}} \quad (4.13)$$

- ❖ Coefficients are estimated for each of the K classes
- ❖ We'll use softmax extensively in neural networks!

Solving the logistic regression cost function (for 2 classes)

$$p(Y = 1|X) = p(X)$$

$$p(Y = 0|X) = 1 - p(X)$$

- ❖ During the training, we'll try to choose β such that, for an observation (x_i, y_i) of our training set:
 - $p(1|x_i)$ will be as close as possible to 1 when $y_i = 1$
 - $p(0|x_i)$ will be as close as possible to 1 when $y_i = 0$
- ❖ We call this approach as the **maximum likelihood**

Likelihood of $(x_1, y_1), \dots, (x_n, y_n)$:

$$\begin{aligned} p(y_1|x_1) \times \dots \times p(y_n|x_n) &= \prod_{i=1}^n p(y_i|x_i) \\ &= \prod_{i:y_i=1} p(x_i) \times \prod_{i:y_i=0} (1 - p(x_i)) \quad (4.5) \\ &= \prod_{i:y_i=1} p(x_i)^{y_i} \times \prod_{i:y_i=0} (1 - p(x_i))^{(1-y_i)} \\ &= \prod_{i=1}^n p(x_i)^{y_i} \times (1 - p(x_i))^{(1-y_i)} \end{aligned}$$

❖ To maximize this likelihood is equivalent to minimize:

$$- \prod_{i=1}^n p(x_i)^{y_i} \times (1 - p(x_i))^{(1-y_i)}$$

Cost function

- ❖ Given the last expression:

$$- \prod_{i=1}^n p(x_i)^{y_i} \times (1 - p(x_i))^{(1-y_i)}$$

- ❖ We can replace $p(X)$ by $f(X)$ and also take log, thus obtaining:

$$J(\beta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log f(x_i) + (1 - y_i) \log(1 - f(x_i))]$$

Cross-entropy cost function

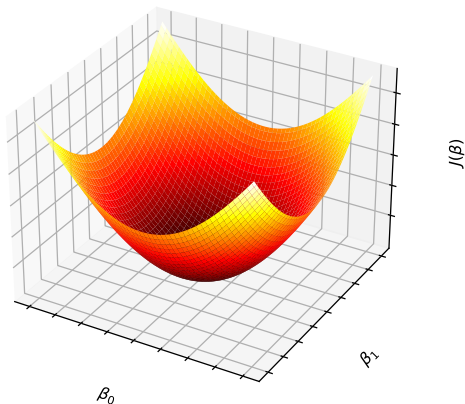
$$J(\beta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log f(x_i) + (1 - y_i) \log(1 - f(x_i))],$$

where $f(X) = \frac{e^{\beta^T X}}{1 + e^{\beta^T X}}$

There is no closed solution for the optimum β !

However, this cost function is **convex**

Cost function



Cross-entropy is **convex** in logistic regression!

LOGISTIC-REGRESSION($\{(x_1, y_1), \dots, (x_n, y_n)\}$)

1 $\beta \leftarrow \mathbf{0}$

2 **while** there is no convergence

3 **do for** each j in $\{0, \dots, p\}$

4 **do** $\beta'_j \leftarrow \beta_j - \alpha \frac{\partial}{\partial \beta_j} J(\beta)$ $\triangleright J$ is the cross entropy

5 $\beta \leftarrow \beta'$ $\triangleright \beta' = \begin{bmatrix} \beta'_0 \\ \vdots \\ \beta'_p \end{bmatrix}$

6 **return** β

Variable normalization must be carried out here as well!